

Spring MVC & Thymeleaf

Arquitectura MVC, SSR, Rutas @{...}, Formularios
y Patrón PRG

Prof. Alejandro Peñaranda

Universidad ICESI – Computación en Internet II

- 1 Sesión 1: Arquitectura MVC, SSR, Sintaxis Base y Layout Modular
- 2 Sesión 1 Live Coding: Config UTF-8, Fragmento Navbar y Listado
- 3 Sesión 2: Rutas `@{...}`, Formularios, Data Binding, Patrón PRG y Controladores
- 4 Sesión 2 Live Coding: Alertas Flash, Servicio, Formulario y CRUD Completo

Sección 01

Arquitectura Spring MVC

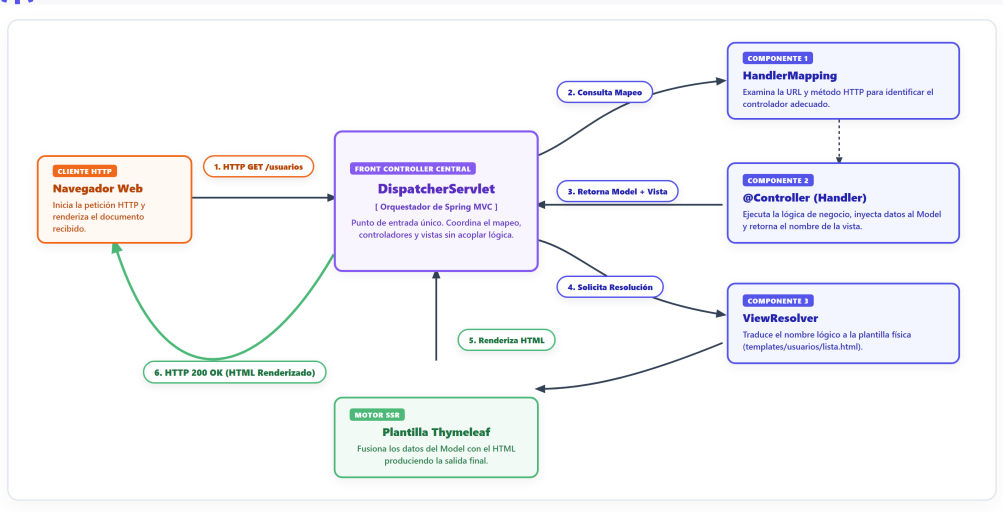
Flujo de Peticiones y Componentes del Núcleo

- **Model (Modelo):** Datos y estado de la aplicación que se transfieren entre el servidor y la interfaz.
- **View (Vista):** Presentación visual de los datos al usuario (HTML procesado en el servidor).
- **Controller (Controlador):** Recibe las peticiones HTTP, invoca los servicios de negocio y elige qué vista renderizar.

Desacoplamiento

El MVC separa completamente la lógica de negocio de la interfaz gráfica, facilitando mantenimiento y testing.

Arquitectura de Spring MVC: Flujo de Ejecución



1. DispatcherServlet: El Front Controller

- **Punto de Entrada Único:** Centraliza y recibe todas las peticiones HTTP dirigidas a la aplicación.
- **Coordinador Central:** No contiene lógica de negocio; orquesta la llamada a los demás componentes.
- **Herencia Servlet:** Extiende de `HttpServlet` y se auto-configura en Spring Boot de forma transparente.

Patrón de Diseño

Implementa el patrón **Front Controller**, evitando que cada ruta requiera un Servlet individual.

- **HandlerMapping:** Mapea la petición HTTP entrante al método del controlador correspondiente.
- **Model** (`ui.Model`): Contenedor clave-valor que transporta datos del Controller a la plantilla.
- **Request Scope:** Los datos viven únicamente durante el ciclo de vida de la petición HTTP.

Uso del Model

Injecta datos en el request:
`model.addAttribute(
 "usuarios", lista);`
Thymeleaf los lee como
`${usuarios}`.

4. ViewResolver: Resolviendo Plantillas

- **Resolución Lógica a Física:** Transforma el String retornado por el Controller en un archivo físico.
- **Prefijo y Sufijo:** Toma "usuarios/lista" y busca en `classpath:/templates/usuarios/lista.html`
- **ThymeleafViewResolver:** Ejecuta el motor Thymeleaf pasando el Model y generando el HTML definitivo.

Convención Spring Boot

Al retornar "usuarios/lista", Spring resuelve automáticamente la ruta sin escribir extensiones .html.

Sección 02

Paradigmas de Renderizado

Server-Side Rendering (SSR) vs Client-Side Rendering (CSR)

Server-Side Rendering (SSR)

- El servidor procesa la lógica, inyecta datos y **construye el HTML final**.
- El navegador recibe un documento HTML listo para renderizar directamente.
- Ejemplo: **Spring Boot MVC + Thymeleaf**.

Client-Side Rendering (CSR)

- El servidor envía un HTML vacío y un archivo JavaScript grande.
- El navegador descarga JS, consulta APIs REST JSON y **construye el DOM en el cliente**.
- Ejemplo: **React, Angular, Vue**.

Flujo Server-Side Rendering (SSR) — Spring Boot + Thymeleaf

RENDERIZADO EN SERVIDOR

PASO 1

Petición HTTP GET

El navegador solicita la URL /usuarios al servidor.



PASO 2

Procesamiento Servidor

El Controller consulta BD y Thymeleaf fusiona datos con el HTML.



PASO 3

Respuesta HTML Completa

El servidor envía el documento HTML final listo con todo su contenido.



PASO 4

Pintado Inmediato

El navegador renderiza la interfaz visual al instante (FCP ultra rápido).

Flujo Client-Side Rendering (CSR) — React / Angular / Vue + REST API

RENDERIZADO EN CLIENTE

PASO 1

Petición Inicial

El navegador solicita la app web al servidor o CDN.



PASO 2

HTML Vacío + Bundle JS

Servidor envía <div id="root"> </div> y archivos JavaScript pesados.



PASO 3

Descarga JS y Fetch API

Navegador ejecuta JS y realiza llamadas asíncronas a /api/usuarios (JSON).



PASO 4

Construcción del DOM

El framework JS procesa el JSON y construye dinámicamente el DOM en el cliente.

@Controller (Spring MVC Tradicional)

- Enfoque: **Server-Side Rendering (SSR)**.
- Retorna un String con el nombre de la vista HTML.
- Interactúa con el Model.
- Ideal para paneles administrativos, portales web y apps monolíticas.

@RestController (Web API REST)

- Enfoque: **Client-Side Rendering (CSR)**.
- Retorna datos crudos en formato **JSON / XML**.
- Anotación compuesta: @Controller + @ResponseBody.
- Ideal para backends que alimentan apps móviles o SPAs (React).

Sección 03

Introducción a Thymeleaf

Motor de Plantillas, Arquitectura Interna y Configuración

Que es Thymeleaf y Cual es su Objetivo?

- **Motor de Plantillas Java:** Diseñado para procesar HTML, XML, CSS y JavaScript en el servidor.
- **Objetivo Principal:** Generar interfaces web dinámicas de forma elegante, manteniendo el código HTML limpio y estandarizado.
- **Reemplazo Moderno de JSP:** Elimina la mezcla desordenada de código Java dentro del HTML, promoviendo alta legibilidad.

Integración Nativa

Thymeleaf se integra al 100 % con el ecosistema Spring (Spring MVC, Form Binding, Spring Security y SpEL).

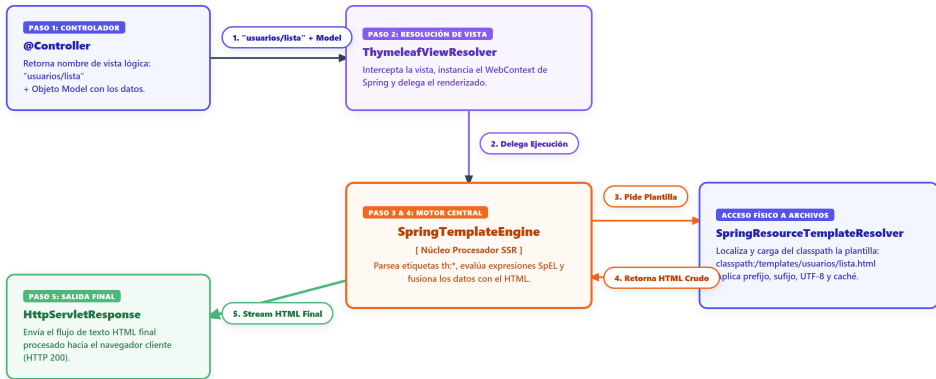
TemplateResolver (Localizador)

- **Rol:** Ubica y carga el archivo físico de la plantilla.
- Configura prefijo (`classpath:/templates/`), sufijo (`.html`), modo (HTML) y política de cache.
- Implementación Spring: `SpringResourceTemplateResolver`.

TemplateEngine (Procesador)

- **Rol:** Núcleo ejecutor que fusiona la plantilla con los datos del `Model`.
- Interpreta atributos `th:*`, evalúa expresiones y aplica dialectos.
- Implementación Spring: `SpringTemplateEngine`.

Flujo Detallado: Interacción Thymeleaf en la Petición



Natural Templating

- Las plantillas son **HTML5 100 % válido**.
- Puedes hacer doble clic en el archivo `.html` y abrirlo en el navegador (`file:///`) como prototipo estático.
- En el servidor, Thymeleaf reemplaza el texto prototipo por los datos reales.

Escape Automático (Anti-XSS)

- `th:text` escapa automáticamente caracteres como `<`, `>`, `&` y comillas.
- Neutraliza intentos de inyección de código malicioso (*Cross-Site Scripting*).
- `th:utext` **no escapa** (usar con precaución).

Estructura de Carpetas y Convenciones en Spring Boot

RAÍZ DEL PROYECTO

src/main/resources/

Contenedor central de configuraciones, activos estáticos y plantillas privadas de Spring Boot.

static/ [Público]

- css/bootstrap.min.css
- js/app.js
- images/logo.png

Acceso Público Directo

Servido directamente por el servidor web sin pasar por ningún `@Controller`.
Referenciado en HTML con sintaxis de contexto: `@{/css/...}`

templates/ [Privado]

- fragments/layout.html
- usuarios/lista.html
- usuarios/formulario.html

Procesamiento Seguro SSR

Archivos protegidos (no accesibles directamente vía URL).
Requieren ser invocados por un `@Controller` y renderizados por el `ViewResolver`.

- 1. Dependencia Starter en `pom.xml`:

```
<!-- pom.xml -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
</dependencies>
```

- 2. Propiedades en `src/main/resources/application.properties`:

```
# Ruta base, extension y soporte UTF-8 en plantillas
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.mode=HTML
spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.cache=false

# Forzar codificación UTF-8 en requests/responses (evita fallos con tildes y n)
spring.servlet.encoding.charset=UTF-8
spring.servlet.encoding.force=true
spring.servlet.encoding.enabled=true
```

```
@Configuration
public class ThymeleafConfig {

    @Bean
    public SpringResourceTemplateResolver templateResolver() {
        SpringResourceTemplateResolver resolver = new SpringResourceTemplateResolver();
        resolver.setPrefix("classpath:/templates/");
        resolver.setSuffix(".html");
        resolver.setTemplateMode(TemplateMode.HTML);
        resolver.setCacheable(false);
        return resolver;
    }

    @Bean
    public SpringTemplateEngine templateEngine(@Qualifier("templateResolver")
                                              SpringResourceTemplateResolver resolver) {
        SpringTemplateEngine engine = new SpringTemplateEngine();
        engine.setTemplateResolver(resolver);
        return engine;
    }
}
```

- **Autoconfiguración Activa:** El starter crea por defecto el bean `defaultTemplateResolver`.
- **El Error:** Si declaras tu propio `templateResolver` manual sin desambiguar, Spring falla con:
- *“expected single matching bean but found 2: templateResolver, defaultTemplateResolver”*.
- **Regla de Oro:** Si usas `application.properties`, no necesitas `ThymeleafConfig`. Si usas la clase manual, agrega `@Qualifier` o `@Primary`.

Mejor Práctica

En Spring Boot se recomienda el enfoque declarativo vía `application.properties` salvo requerimiento avanzado.

Sección 04

Sintaxis Esencial de Thyme- leaf

Atributos de Lectura, Expresiones $\$\{...\}$ y $@\{...\}$, Condicionales e Iteración

- **Namespace XML:** Se declara en la cabecera como
`xmlns:th="http://www.thymeleaf.org".`
- **Filosofía:** Convierte cualquier HTML5 estático en una plantilla viva sin romper su visualización en navegadores.
- **Enfoque Sesión 1 (Lectura y Listados):**
 1. **Expresiones** `${...}` y `@{...}`: Lectura de datos del Model y construcción de URLs.
 2. **Atributos** `th:text`, `th:each`, `th:if` e `iterStat` para tablas y vistas dinámicas.

Ejecución en el Servidor

En tiempo de ejecución, Thymeleaf evalúa las expresiones, reemplaza los valores prototipo y elimina los prefijos `th:.`

$\${...}$: Variables del Model

- Evalúa atributos del Model.
- Acceso a campos: $\${u.correo}$.
- Soporta operadores y ternarios.

$@{...}$: Enlaces y URLs

- Resuelve el *context-path*.
- **Path Vars:**
 $@{/usuarios/{id}(id=\${u.id})}$.
- **Query Params:**
 $@{/usuarios(rol='DOC')}$.

```
<!-- 1. Variables del Model -->
<h2 th:text="\${titulo}">Titulo</h2>
<p th:text="\${usuario.nombre} + ' ' + usuario.apellido">
  Juan Perez
</p>

<!-- 2. Path Variable y Context Path -->
<a th:href="@{/usuarios/detalle/{id}(id=\${u.id})}">
  Ver Detalle
</a>

<!-- 3. Query Parameter -->
<a th:href="@{/usuarios(estado='ACTIVO')}">
  Filtrar Activos
</a>
```

th:text (Escapado Seguro)

- Convierte caracteres especiales (<, >, &, ") en entidades HTML.
- **Seguridad:** Protege la app contra ataques de inyección *Cross-Site Scripting* (XSS).

th:utext (Unescaped / Crudo)

- Renderiza etiquetas HTML directamente en el DOM.
- **Precaución:** Usar solo con contenido confiable y previamente sanitizado.

```
// En el Controlador Spring:  
model.addAttribute("msg",  
    "<strong>Bienvenido</strong>");
```

```
<!-- En la Plantilla HTML: -->  
  
<!-- 1. Escapado (Muestra texto literal) -->  
<p th:text="${msg}">Prototipo</p>  
<!-- Salida: &lt;strong&gt;Bienvenido... -->  
  
<!-- 2. Sin escapar (Renderiza en negrita) -->  
<p th:utext="${msg}">Prototipo</p>  
<!-- Salida: <strong>Bienvenido</strong> -->
```

Iteración de Colecciones

- Recorre colecciones: elem, stat :
\${lista}.

Propiedades de iterStat

- index / count: Índice base 0 / base 1.
- size: Total de elementos.
- even / odd: Fila par o impar.
- first / last: Primer o último elemento.

```
<table>
  <thead>
    <tr>
      <th>#</th> <th>Nombre</th> <th>Estado</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="u, stat : ${usuarios}">
      <td th:text="${stat.count}">1</td>
      <td th:text="${u.nombre}">Juan</td>
      <td>
        <span th:if="${stat.first}">Primero</span>
      </td>
    </tr>
  </tbody>
</table>
```

Evaluación Booleana

- **th:if:** Renderiza la etiqueta si la condición es true, número no cero o texto no vacío.
- **th:unless:** Opuesto de if; renderiza si la condición es false o null.

```
<!-- Condicionales if / unless -->
<span th:if="${user.active}">Activo</span>
<span th:unless="${user.active}">Inactivo</span>

<!-- Evaluacion Switch / Case -->
<div th:switch="${user.rol}">
  <p th:case="'ADMIN'">Panel Administrador</p>
  <p th:case="'DOCENTE'">Gestion de Cursos</p>
  <p th:case="*">Acceso Estudiante</p>
</div>
```

Bifurcación Múltiple

- **th:switch & th:case:** Evalúa múltiples opciones.
- **th:case="*"** actúa como la opción por defecto (*default*).

Enlaces, Recursos y Atributos Dinámicos: th:href, th:src, th:attr

th:href & th:src

- Gestionan hipervínculos, hojas de estilo e imágenes asegurando resolución context-aware.

th:attr

- Permite modificar o crear cualquier atributo HTML genérico (data-*, title, alt, target).

```
<!-- Enlace y Recurso Estatico -->
<a th:href="@{/usuarios}">Volver</a>


<!-- Atributos Arbitrarios y HTML5 Data -->
<button th:attr="data-id=${user.id}, title=${user.nombre}">
  Ver Detalle
</button>
```

#{...}: Message Expressions (i18n)

- Lee textos externalizados desde `messages.properties`.
- Permite soporte multi-idioma automático según la configuración regional (*Locale*) del navegador.
- Soporta paso de parámetros posicionales: `#{msg.key(param1, param2)}`.

```
# messages.properties
welcome.banner=Sistema Academico ICESI
usuarios.titulo=Listado de Usuarios Registrados
usuarios.total=Total encontrados: {0}
```

```
<!-- En la Plantilla HTML: -->
<h1 th:text="#{welcome.banner}">Titulo</h1>
<p th:text="#{usuarios.total(${usuarios.size()})}">
    Total encontrados: 5
</p>
```

- **Definición (`th:fragment`):** Declara un bloque reusable en un archivo parcial (ej. `layout.html`).
- **Reutilización Global:** Elimina la duplicación de código en cabeceras, menús de navegación y pies de página.
- **Mantenimiento Centralizado:** Modificar la barra de navegación en el fragmento actualiza automáticamente toda la aplicación.

Sintaxis de Invocación

```
th:replace="
~{fragments/layout ::
mainNavbar} "
```

th:replace (Sustituto)

- **Reemplaza por completo** la etiqueta anfitriona por la etiqueta del fragmento.
- Ideal para `<header>`, `<nav>` y bloques modulares.

th:insert (Hijo)

- **Inserta** el fragmento dentro de la etiqueta contenedora sin eliminarla.

Sintaxis de Fragmento

- `~{archivo :: nombreFragmento}`

```
<!-- 1. Definicion en templates/fragments/layout.html -->
<nav th:fragment="mainNavbar">
  <a th:href="@{/usuarios}">Portal ICESI</a>
  <a th:href="@{/usuarios/nuevo}">Nuevo</a>
</nav>

<!-- 2. Uso en templates/usuarios/lista.html -->
<!-- th:replace elimina <header> y coloca <nav> -->
<header th:replace="~{fragments/layout :: mainNavbar}">
</header>
```

Listado Modular

Thymeleaf + Layouts

“La interfaz cobra vida: fragmentos reusables, controlador desacoplado y renderizado SSR 100 % dinámico y seguro.”

Sección 05

Enlaces Dinámicos y Rutas con @{...}

Path Variables, Query Params y Context-Path

Context-Path y Recursos

- @{...} antepone automáticamente el *context-path* del servidor.
- Rutas a CSS, JS o imágenes siempre funcionales.

Parámetros de Ruta (Path Variables)

- Sintaxis:
@{/ruta/{id}(id=\${u.id})}.
- Sustituye {id} de forma segura sin concatenar cadenas manualmente.

```
<!-- 1. Enlace a CSS/JS en /static/css/ -->
<link rel="stylesheet" th:href="@{/css/styles.css}" />

<!-- 2. Path Variables dinamicas -->
<!-- Genera: /usuarios/editar/42 -->
<a th:href="@{/usuarios/editar/{id}(id=${u.id})}">
    Editar Usuario
</a>

<!-- Genera: /usuarios/desactivar/42 -->
<a th:href="@{/usuarios/desactivar/{id}(id=${u.id})}">
    Cambiar Estado
</a>
```

Query Parameters con Tuplas

- Para filtros, búsquedas, ordenamiento o paginación en la URL.
- Sintaxis: @{/ruta(param1=val1, param2=val2)}.
- Thymeleaf escapa caracteres automáticamente (%20).

Combinación Path + Query

- Permite combinar variables de ruta y query params en una misma expresión de tupla.

```
<!-- 1. Query Params simples -->
<!-- Genera: /usuarios?estado=activo&page=1 -->
<a th:href="@{/usuarios(estado='activo', page=1)}">
    Ver Usuarios Activos
</a>

<!-- 2. Path Variable + Query Param combinado -->
<!-- Genera: /usuarios/42/historial?seccion=cursos -->
<a th:href="@{/usuarios/{id}/historial(id=${u.id}, seccion='
    cursos')}">
    Ver Historial
</a>
```

Path Variable (@PathVariable)

- **Propósito:** Identificar un recurso único y específico en el sistema.
- **Obligatoriedad:** Obligatorio. Si falta, la ruta no existe (HTTP 404).
- **Semántica REST:** Define jerarquías:
`/cursos/{cursoId}/estudiantes/{id}`
- **Ejemplo:** Editar o ver detalle de un usuario (`/usuarios/42`).

Query Parameter (@RequestParam)

- **Propósito:** Modificar, filtrar, paginar u ordenar una colección.
- **Obligatoriedad:** Opcional. Soporta valores por defecto (`defaultValue`).
- **Semántica REST:** Modificador no canónico de la vista:
`/usuarios?rol=DOCENTE&page=2`
- **Ejemplo:** Filtro por estado o búsqueda.

Regla de Oro: ¿Identifica el **QUÉ** recurso es? → **Path Variable**. ¿Modifica el **CÓMO** se presenta la lista? → **Query Param**.

Sección 06

Formularios y Data Binding

th:object, @ModelAttribute y los 3 Efectos de th:field

Ciclo Bidireccional de Datos

- **1. GET (Backend $\rightarrow$ Vista):**
Controller inyecta el modelo:
`addAttribute("usuario", u).`
- **2. HTML (Plantilla Form):**
`th:object="${usuario}"` vincula
el form y `th:field` mapea
controles.
- **3. POST (Vista $\rightarrow$ Backend):**
`@ModelAttribute("usuario")`
deserializa y puebla la entidad Java.

```
// 1. GET: Inyecta el objeto en el Model
@GetMapping("/nuevo")
public String mostrarForm(Model model) {
    model.addAttribute("usuario", new Usuario());
    return "usuarios/formulario";
}

// 3. POST: Deserializa el objeto enviado
@PostMapping("/guardar")
public String guardar(
    @ModelAttribute("usuario") Usuario usuario) {
    usuarioService.guardar(usuario);
    return "redirect:/usuarios";
}
```

Objeto Respaldo (th:object)

- Vincula el `<form>` a la entidad Java inyectada en el Model.

Selección Relativa ($\ast\{...\}$)

- $\ast\{\text{nombre}\}$ equivale a `\${usuario.nombre}`.
- Acceso directo a propiedades del objeto activo en el formulario.

Preservación de ID

- `type="hidden"` preserva el ID para diferenciar Creación de Edición.

```
<form th:action="@{/usuarios/guardar}"
      th:object="${usuario}" method="post">

  <!-- Campo oculto para ID en edicion -->
  <input type="hidden" th:field="*{id}" />

  <div>
    <label for="nombre">Nombre:</label>
    <input type="text" th:field="*{nombre}" required />
  </div>

  <div>
    <label for="apellido">Apellido:</label>
    <input type="text" th:field="*{apellido}" required />
  </div>

  <button type="submit">Guardar</button>
</form>
```

Automatización de 3 Atributos

- **name:** Clave en el payload HTTP POST.
- **id:** Vincula semánticamente con `<label for="...">`.
- **value:** Precarga el valor en modo edición.

Soporte Multi-Control

- **Checkbox:** Gestiona booleanos (checked).
- **Select:** Asigna el selected automático.

```
<!-- 1. Input de Texto y Correo -->
<label for="correoInstitucional">Email:</label>
<input type="email" th:field="*{correoInstitucional}" />

<!-- 2. Checkbox Booleano -->
<label for="active">
  <input type="checkbox" th:field="*{active}" />
  Usuario Activo
</label>

<!-- 3. Parametro Separado para Relaciones (Roles) -->
<label for="rol">Rol Inicial:</label>
<select id="rol" name="nombreRol">
  <option value="ESTUDIANTE" selected>Estudiante</option>
  <option value="PROFESOR">Profesor</option>
  <option value="ADMIN">Administrador</option>
</select>
```

Sección 07

Controladores Web y Patrón PRG

Parámetros, Procesamiento POST y Redirección 302

Captura de Parámetros en Controller: @PathVariable y @RequestParam

@PathVariable (Rutas RESTful)

- Variables embebidas en el path (ej. /usuarios/editar/{id}).
- Operaciones individuales sobre un recurso por su ID primario.

@RequestParam (Query Strings)

- Parámetros de consulta (?clave=valor).
- Soporta defaultValue y parámetros opcionales (required=false).

```
// 1. Path Variable: GET /usuarios/editar/42
@GetMapping("/editar/{id}")
public String mostrarFormularioEditar(
    @PathVariable("id") Long id, Model model) {
    Usuario usuario = userService.obtenerPorId(id);
    model.addAttribute("usuario", usuario);
    return "usuarios/formulario";
}

// 2. Query Param: GET /usuarios/filtrar?estado=activo
@GetMapping("/filtrar")
public String filtrarUsuarios(
    @RequestParam(value = "estado",
        defaultValue = "todos") String est,
    Model model) {
    model.addAttribute("usuarios",
        userService.filtrarPorEstado(est));
    return "usuarios/lista";
}
```

Peligro de Reenvío (F5)

- Retornar HTML directo tras un POST causa que F5 repita la mutación, duplicando registros en la base de datos.

Fase 1 (POST: Mutación)

- Guarda en BD y retorna **HTTP 302** (redirect:/usuarios).

Fase 2 (GET: Lectura Segura)

- Solicita GET /usuarios (idempotente). Al presionar F5, sólo se repite la lectura segura.

```
<!-- CICLO DEL PATRON PRG -->

1. POST /usuarios/guardar
   -> Service guarda en BD
   -> Retorna: "redirect:/usuarios" (HTTP 302)

2. NAVEGADOR recibe 302
   -> Solicita automaticamente GET /usuarios

3. GET /usuarios (Lectura Segura)
   -> Service.listarUsuariosActivos()
   -> Retorna: "usuarios/lista" (HTTP 200 OK)

[!] Al presionar F5 en el navegador:
    Solo se reejecuta GET /usuarios.
    El POST NUNCA se repite.
```

1. Controller: addFlashAttribute

- Tras un HTTP 302, los datos del Model se pierden.
- addFlashAttribute viaja en HttpSession temporalmente y se auto-elimina tras el GET.

```
@PostMapping("/guardar")
public String guardar(
    @ModelAttribute("usuario") Usuario u,
    RedirectAttributes flash) {
    usuarioService.guardar(u);
    flash.addFlashAttribute(
        "exito", "Usuario guardado!");
    return "redirect:/usuarios";
}
```

2. Plantilla: Fragmento de Alertas

```
<!-- templates/fragments/layout.html -->
<div th:fragment="alerts">
    <div th:if="${exito}">
        <span th:text="${exito}">Exito</span>
    </div>
    <div th:if="${error}">
        <span th:text="${error}">Error</span>
    </div>
</div>

<!-- Uso en lista.html o formulario.html -->
<div th:replace="~{fragments/layout :: alerts}">
</div>
```

Sección 08

Utilidades de Expresión y Scripts

Objetos # y JavaScript Inline

Objetos de Utilidad de Expresión: #strings, #numbers, #temporals

Objetos Predefinidos de Ayuda

- #strings: toUpperCase, toLowerCase, contains, substring, isEmpty.
- #numbers: Formato decimal y moneda (formatDecimal).
- #temporals / #dates: Formateo de fechas Java 8 (LocalDate, LocalDateTime).
- #lists: size, isEmpty, contains.

```
<!-- 1. Cadenas de Texto -->
<p th:text="${#strings.toUpperCase(u.nombre)}"></p>
<p th:if="${#strings.contains(u.correoInstitucional, 'icesi.edu.co')}">
    Usuario Institucional
</p>

<!-- 2. Formato de Fechas Java 8 -->
<span th:text="${#temporals.format(u.fechaCreacion, 'dd/MM/yyyy')}">
    01/01/2026
</span>

<!-- 3. Formato Numerico y Colecciones -->
<span th:text="${#numbers.formatDecimal(u.promedio, 1, 2)}">
    4.50
</span>
<p th:if="${#lists.isEmpty(usuarios)}">Sin registros</p>
```

Recursos Estáticos

- Referenciados con
@{/css/styles.css},
@{/js/app.js} garantizando
resolución context-aware.

JavaScript Inline (th:inline)

- th:inline="javascript": Inyecta variables del Model en scripts cliente.
- Sintaxis /*[[\${...}]]*/
'fallback' asegura compatibilidad como prototipo estático en navegador.

```
<!-- Enlace a hoja de estilos -->
<link rel="stylesheet" th:href="@{/css/styles.css}">

<!-- Inyección de datos Java en JavaScript -->
<script th:inline="javascript">
    /* Thymeleaf evalúa la expresión entre comentarios */
    let nombreUsuario = /*[[${usuario.nombre}]]*/ 'Prototipo';
    let totalActivos = /*[[${totalUsuarios}]]*/ 0;

    console.log("Usuario activo:", nombreUsuario);
</script>
```

- **1. Seguridad XSS por Defecto:** Emplear siempre `th:text`; reservar `th:utext` sólo para HTML sanitizado.
- **2. Natural Templating:** Diseñar plantillas válidas estáticamente en navegador y dinámicamente en Spring Boot.
- **3. Separación MVC:** El Controller sólo orquesta; el Service gestiona el negocio y la vista presenta datos.
- **4. Idempotencia y Patrón PRG:** Todo `@PostMapping` debe redirigir con HTTP 302 y usar mensajes flash.

Próximos Pasos

En la Semana 9 integraremos **Spring Security** con Thymeleaf para proteger vistas y botones según el rol autenticado.

CRUD 100% Funcional

Thymeleaf + Spring Boot MVC

*“Hemos construido una aplicación web robusta, modular, segura
y fácil de mantener aplicando SSR y buenas prácticas de
arquitectura.”*